Complexité et réduction

Imed Kacem

1 SAT, 3-SAT et CIRCUIT-SAT

1.1 Rappels de définitions

SAT

1. **En entrée :** un ensemble de variables propositionnelles $X = \{x_1, \ldots, x_n$ t.q. $x_j \in \mathbb{B}\}$ et une formule propositionnelle $F = c_1 \wedge c_2 \wedge \cdots \wedge c_k$ en forme normale conjonctive où chaque clause c_i, $1 \leq i \leq k$, est une disjonction de littéraux (un littéral est soit une variable x_j, soit sa négation $\neg x_j$).
2. **Questions :** existe-t-il une affectation de valeurs de vérité pour X de sorte que la formule F soit satisfiable ?

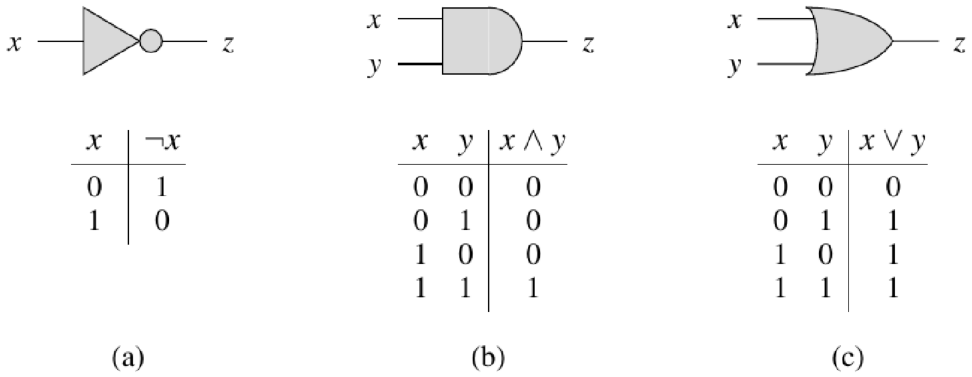
3-SAT

1. **En entrée :** un ensemble de variables propositionnelles $X = \{x_1, \ldots, x_n$ t.q. $x_j \in \mathbb{B}\}$ et une formule propositionnelle $F = c_1 \wedge c_2 \wedge \cdots \wedge c_k$ en forme normale conjonctive où chaque clause c_i, $1 \leq i \leq k$, est une disjonction de 3 littéraux (un littéral est soit une variable x_j, soit sa négation $\neg x_j$).
2. **Question :** existe-t-il un assignement de valeurs de vérité pour X de sorte que la formule F soit satisfiable ?

CIRCUIT-SAT
C'est un problème de *décision* qui consiste à vérifier l'existence d'une entrée $(x_1, x_2, \ldots, x_n)$ valide pour mettre à 1 la sortie d'un circuit combinatoire.
Illustration.

1.2 Questions

1. Montrez que **3-SAT** $\leq$ **SAT**.
2. Montrez que **SAT** $\leq$ **3-SAT**.
3. Montrez que **SAT** $\leq$ **CIRCUIT-SAT**.
4. Montrez que **CIRCUIT-SAT** $\leq$ **SAT** et **SAT** $\leq$ **CIRCUIT-SAT**.
5. Montrez que **3-SAT** $\leq$ **CLIQUE**.
6. Montrez que **CLIQUE** $\leq$ **COUVERTURE-SOMMETS**.



x	$\neg x$
0	1
1	0

(a)

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

(b)

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

(c)

FIGURE 1 – Les portes des circuits logiques.

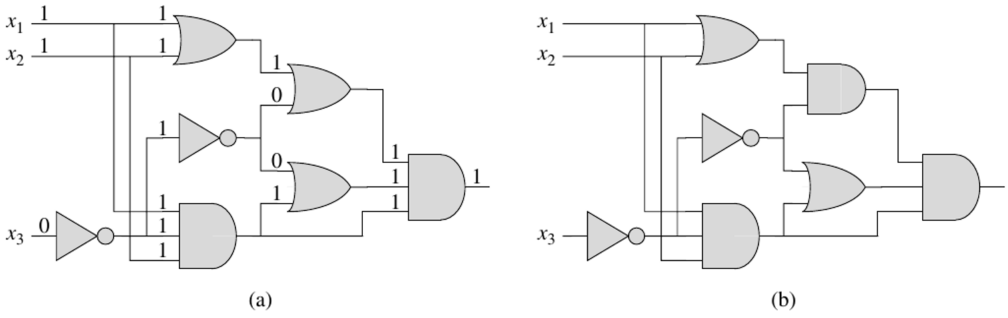


FIGURE 2 – **(a)** Un circuit satisfiable ; **(b)** un circuit non satisfiable.

SAT.

Instance : n : nb variables
m : nb clauses
x_i : variable
c_i : clause

en forme normale conjective (CNF)

$$c_i = x_1 \vee \overline{x_2} \vee x_3 \vee \dots \vee x_i$$

de manière générale : $c_i = l_{i1} \vee l_{i2} \vee l_{i3} \vee \dots$

La formule à satisfaire est $F = \bigwedge_{i=1}^{n} c_i$

Existe-t-il une affectation de valeurs V/F pour satisfaire F ?

↳ $(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_2 \vee x_3) \wedge (x_1 \vee x_2)$

$\Rightarrow S = \{x_1, x_2\}$.

3-SAT : Instance identique à SAT mais chaque clause contient exactement 3 littéraux, $|c_i| = 3$.

Le problème est identique à SAT.

On note $\begin{cases} SAT \leqslant_p 3SAT \\ 3SAT \leqslant_p SAT \end{cases}$

$\Rightarrow SAT =_p 3SAT$

$A \leqslant_p B$: B est au moins aussi difficile que le problème A.
A peut être réduit d'une façon polynomiale par rapport à B.

⇒ Comme 3SAT $\subseteq$ SAT, 3SAT $\leq_p$ SAT.
Il faut prouver SAT $\leq_p$ 3SAT

↳ Prouver SAT $\leq_p$ 3SAT

On pourrait avoir des chgts de variable.
Si on a un pb de SAT : $\begin{cases} n : \text{nb } x_i \\ m : \text{nb } c_i \\ x_i \in \{0,1\} \\ c_i \text{ , } |c_i| > 0 \end{cases}$

On dispose de différents cas pour transformer chaque c_i.

- $\begin{cases} |c_i| = 1 \\ |c_i| = 2 \\ |c_i| = 3 \\ |c_i| > 3 \end{cases}$

Si $|c_i| = 1$: $c_i = (l_j)$

On pourrait penser à $c'_i = l_j \vee l_j \vee l_j$, pas possible car littéraux uniq / clause

On peut aussi augmenter le nb de clauses.
On ajoute 2 variables et les combin. lin. associées

$l_j = l_j \wedge (\bar{x}_1 \vee x_1) \wedge (\bar{x}_2 \vee x_2)$

$$l_j = (l_j \vee x_1 \vee x_2) \wedge (l_j \vee x_1 \vee \bar{x}_2) \vee (l_j \vee \bar{x}_1 \vee x_2) \vee (l_j \vee \bar{x}_1 \vee \bar{x}_2)$$

Peu importe (x_1, x_2), pour que c'_i soit valide il faut que l_j soit vrai.

Si $|C_i| = 2$: $c_i = l_j \vee l_k$

$$(l_j \vee l_k) = (l_j \vee l_k) \wedge (x_1 \vee \bar{x}_1)$$

$$(l_j \vee l_k) = (l_j \vee l_k \vee x_1) \wedge (l_j \vee l_k \wedge \bar{x}_1)$$

Même argument qu'au dessus.

Si $|C_i| = 3$: rien à faire

Si $|C_i| > 3$: $c_i = (l_{j_1} \vee l_{j_2} \vee \dots \vee l_{j_k}) \mid k > 3$.

$$\Rightarrow l_{i1} \vee l_{i2} \vee l_{i3} \vee \dots \vee l_{ik}$$

$$= (l_{i1} \vee l_{i2} \vee x_1) \wedge (\bar{x}_1 \vee l_{i3} \vee \dots \vee l_{ik})$$

Rajout d'une variable

$$= (l_{i1} \vee l_{i2} \vee x_{i1}) \wedge (\bar{x}_{i1} \vee l_{i3} \vee x_{i2}) \wedge (\bar{x}_{i2} \vee l_{i4} \vee \dots \vee l_{ik})$$

intermédiaire. Chq x_i a une contre paire de la clause suivante

$$= (l_{i1} \vee l_{i2} \vee x_{i1}) \wedge (\bar{x}_{i1} \vee l_{i3} \vee x_{i2}) \wedge (\bar{x}_{i2} \vee l_{i4} \vee \bar{x}_3) \wedge (\dots)$$
$$\wedge (\bar{x}_{i,k-3} \vee l_{i,k-1} \vee l_{ik})$$

Instance de 3 SAT créée a $\leq m.n$ clauses
$\leq m.n + n$ variables.

Ça reste polynomial par rapport à l'instance initiale

Ainsi, $SAT \leq_p 3SAT$ et 3SAT est NP-difficile.

Clique Dans un graphe, $G = (V, E)$

Quest° : clique maximale, trouver le plus gd $K_n \in G \mid n \geq w$?

K_2 : —

K_3 : △

K_4 : ⊠

K_5 : ⭐

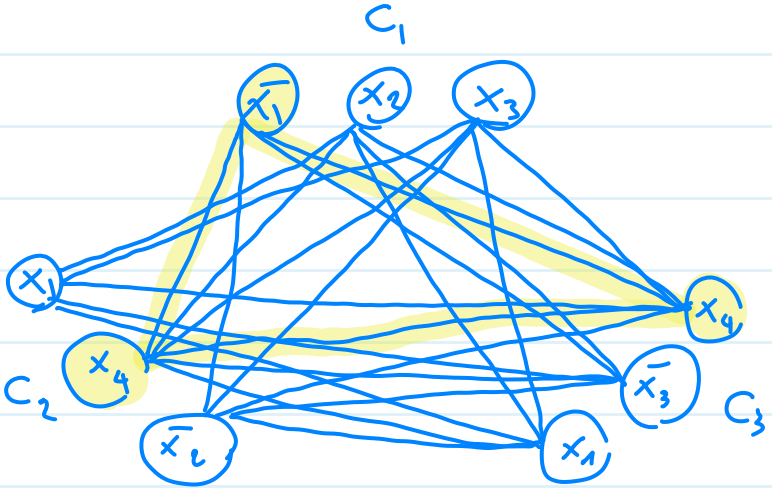
K_n : $\frac{n(n-1)}{2}$ arêtes.

$\Rightarrow 3SAT \leq_p CLIQUE$

Appliquer les cliques à 3SAT :

Ex : $(\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_4 \vee \bar{x}_2) \wedge (x_1 \vee \bar{x}_3 \vee x_4)$

c_1 c_2 c_3



On ne lie pas x_i et $\bar{x}_i$.

Construction :

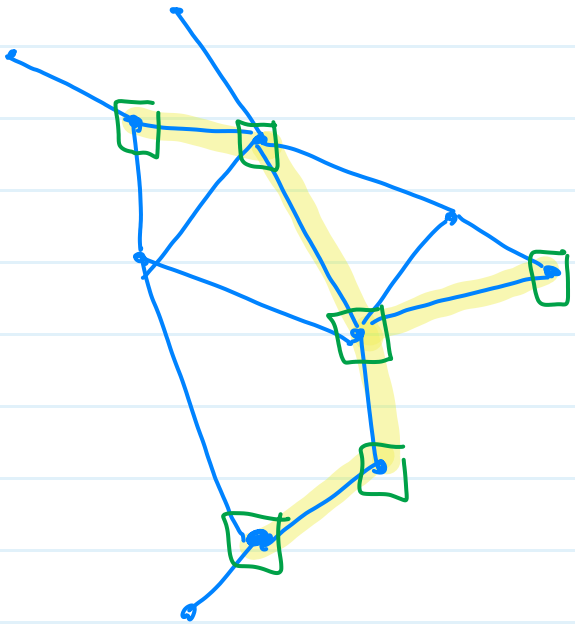
- Pour chq littéral, créer un sommet à G
- Créer des arêtes :
  - → R_1 : Une arête peut lier uniquement des sommets de classe diff.
  - → R_2 : On ne peut pas lier x_i et $\bar{x_i}$.
- $K = m$ = nb de clauses.

$\exists\ K_{n,1}$ $n \geq m$ $\Leftrightarrow$ F satisfaisable.

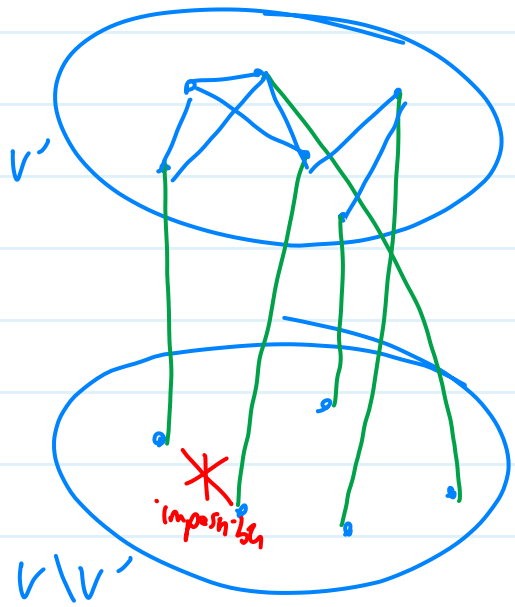
COUVERTURE - SOMMETS

Instance : un graphe $G = (V, E)$
k entier, $k \leq |V|$.

Quest° : Y a-t-il $V' \subseteq V$ | $|V'| = K \wedge \forall e = (u,v) \in E$, on a $(u \in V' \text{ ou } v \in V')$



et



Stable car aucune arête

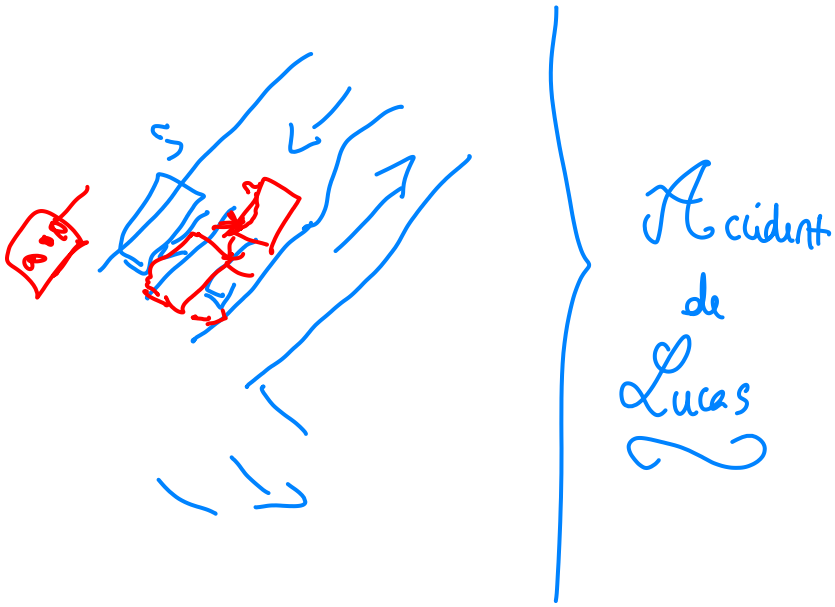
Les stables sont les inverses des cliques.
On peut créer le graphe:

$\overline{G} = (V, \overline{E})$ tq $\overline{E}$ soit l'inverse de E.

Ainsi, $K' = n - K$

1.3 Supplément proposé par Imed Kacem

1. Montrez que **CYCLE-HAMILTONIEN** $\leq$ **COMMISE-VOYAGEUSE**. (déjà traité lors des premiers Tds)
2. On considère le problème $2\|C_{\max}$
   (a) Montrez que ce problème est polynomial si le codage est *unaire*.
   (b) Montrez que ce problème est dans NPC si le codage est *binaire*.
3. On considère le problème $P\|C_{\max}$ (avec un nombre quelconque de machines parallèles)
   On admet que le problème n'a pas d'algorithme exact *pseudo-polynomial*.
   Montrez que le problème n'admet pas de FPTAS sauf si P = NP.



1.4 Réponses

1.4.1 3_SAT $\leq$ SAT

3-sat est un ca s partuclier de **sat** ; une formule propositionnelle en forme normale conjonctive de clauses disjonstives de 3 litéraux est a fortiori une formule propositionnelle en forme normale conjonctive de clauses disjonstives de litéraux (*en nombre quelconque, ici fixé à* 3)

1.4.2 SAR $\leq$ 3-SAT

SAT étant une généralisation de **3-SAT** la question est : *Est-ce que tout formule propositionnelle en forme normale conjonctive de clauses disjonctives de littéraux (en nombre quelconque) peut être réécrite en forme normale conjonctive de clauses disjonctives de* 3 *littéraux ?*
On écrit la formule propositionnelle de **SAT** :

$$F = \wedge_{i=1}^{k} c_i \quad \text{où} \quad c_i = \vee_{j=1}^{m_i} l_{i,j} \quad \text{et} \quad l_{i,j} = x_o \text{ ou } l_{i,j} = \neg x_o \text{ avec } x_o \in X$$

— $\mathbf{m_i = 3}$: la clause est sous forme **3-SAT**.
— $\mathbf{m_i = 2}$: on prend une variable propositionnelle y et on remplace la clause $c_i = l_{i,1} \vee l_{i,2}$ par les 2 clauses $c_{i,1}$ et $c_{i,2}$ à 3 littéraux suivantes :

$$c_{i,1} = l_{i,1} \vee l_{i,2} \vee y \quad \text{et} \quad c_{i,2} = l_{i,1} \vee l_{i,2} \vee \neg y$$

— $\mathbf{m_i = 1}$: on prend alors 2 variables propositionnelles y_1 et y_2 et on remplace chaque clause $c_i = l_{i,1}$ par les 4 clauses $c_{i,1}$, $c_{i,2}$, $c_{i,3}$, et $c_{i,4}$ à 3 littéraux suivantes :

$$c_{i,1} = l_{i,1} \vee y_1 \vee y_2 \quad \text{et} \quad c_{i,2} = l_{i,1} \vee \neg y_1 \vee y_2 \quad \text{et} \quad c_{i,3} = l_{i,1} \vee y_1 \vee \neg y_2 \quad \text{et} \quad c_{i,4} = l_{i,1} \vee \neg y_1 \vee \neg y_2$$

En effet, quand c_i est satisfiable toutes les clauses $c_{i,j}$ avec $1 \leq j \leq 4$ doivent l'être également et ce quelque soit les valeurs y_1 et y_2. Si y_1 et y_2 prennent la valeur $Faux$ alors les clauses $c_{i,j}$ avec $2 \leq j \leq 4$ sont satisfaites quelque soit la valeur de $l_{i,1}$ et seule la clause $c_{i,1}$ est satisfaite grâce à $l_{i,1}$. En observant toutes les combinaisons possibles de valeurs pour y_1 et y_2 on constate qu'il y a toujours trois clauses $c_{i,j}$ satisfaites quelque soit la valeur de $l_{i,1}$ et que la quatrième clause est satisfaite grâce à $l_{i,1}$. Autrement dit :

$$c_i \iff \bigwedge_{j=1}^{4} c_{i,j}$$

— $\mathbf{m_i > 3}$: on prend $p = m_i - 3$ variables propositionnelles y_1, y_2,
$ldots$, y_p et on remplace la clause c_i par les $p+1$ clauses à 3 littéraux suivantes :

$$\begin{aligned} c_{i,1} &= x_1 \vee x_2 \vee y_1 \\ c_{i,j} &= \neg y_{j-1} \vee x_{j+1} \vee y_j, \quad \forall 1 < j \leq m_i - 3 \\ c_{i,m_i-2} &= \neg y_{m_i-3} \vee x_{m_i-1} \vee x_{m_i} \end{aligned}$$

Ainsi :

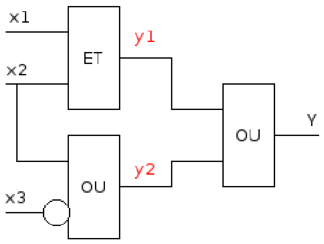
$$c_i \iff \bigwedge_{j=1}^{p+1} c_{i,j}$$

1.4.3 SAT $\leq$ CIRCUIT-SAT

Évident

1.4.4 CIRCUIT-SAT $\leq$ SAT

Prenons l'exemple ci-après auquel en plus des entrées x_1, x_2 et de la sortie y on ajoute les variables intermédiaires y_1 et y_2 représentant les sorties des deux premiers connecteurs.



On peut alors décrire le circuit comme une conjonction de connecteurs où les connecteurs ne sont autre chose que des *termes* de 3 littéraux.

$$f = \underbrace{(Y \iff (y_1 \vee y_2))}_{c_1} \wedge \underbrace{(y_1 \iff (x_1 \wedge x_2))}_{c_2} \wedge \underbrace{(y_2 \iff (x_2 \vee \bar{x_3}))}_{c_3}$$

Cependant ce n'est pas une *forme normale conjonctive*. Il faut transformer chaque clause. Si un connecteur a plus de 2 entrées, on peut le substituer par une cascade de connecteurs à 2 entrées et une sortie.

En établissant la table de vérité de chaque connecteur c_i de f et en retenant les valeurs des littéraux les invalidant, on obtient pour chacun d'eux une *forme normale disjonctive* :

$$\neg c_1 = (Y \wedge \neg y_1 \wedge \neg y_2) \vee (\neg Y \wedge y_1 \wedge \neg y_2) \vee (\neg Y \wedge \neg y_1 \wedge y_2) \vee (\neg Y \wedge y_1 \wedge y_2)$$
$$\neg c_2 = (\neg x_1 \wedge \neg x_2 \wedge y_1) \vee (\neg x_1 \wedge x_2 \wedge y_1) \vee (x_1 \wedge \neg x_2 \wedge y_1) \vee (x_1 \wedge x_2 \wedge \neg y_1)$$
$$\neg c_3 = (\neg x_2 \wedge \neg x_3 \wedge \neg y_2) \vee (\neg x_2 \wedge x_3 \wedge y_2) \vee (x_2 \wedge \neg x_3 \wedge \neg y_2) \vee (x_2 \wedge x_3 \wedge \neg y_2)$$

Ce qui permet d'écrire :

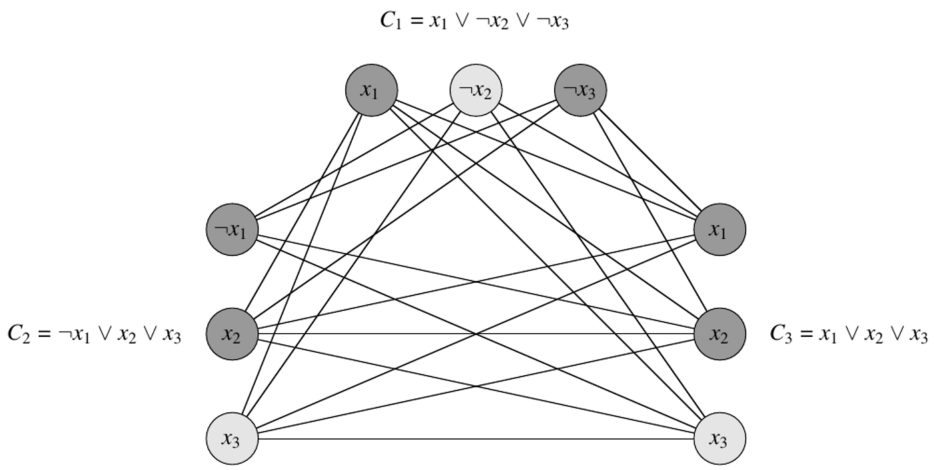
$$\neg f = \neg c_1 \vee \neg c_2 \vee \neg c_3$$

Avec un peu de manipulation on arrive à une forme normale conjonctive de f.

1.4.5 3-SAT $\leq$ CLIQUE

Tout est dit dans la figure qui représente le graphe correspondant à la formule :

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$



1.4.6 CLIQUE $\leq$ COUVERTURE-SOMMETS

Rappel. une couverture de sommets d'un graphe $G = (S, A)$ est un ensemble de sommets $S' \subseteq S$ qui couvre toute les arêtes de A

Tout est dit dans la figure où le graphe G à gauche contient $|S| = 6$ sommets dont $k = 4$ forment la clique unique et donc maximale. Le graphe $\bar{G}$ à droite est son complémentaire au sens des arcs ; il contient les mêmes sommets mais l'ensemble $\bar{A}$ de ses arcs est le dual de l'ensemble A des arcs de G : tout couple (u, v) de sommets de S^2 est soit un élément de A soit un élément de $\bar{A}$.

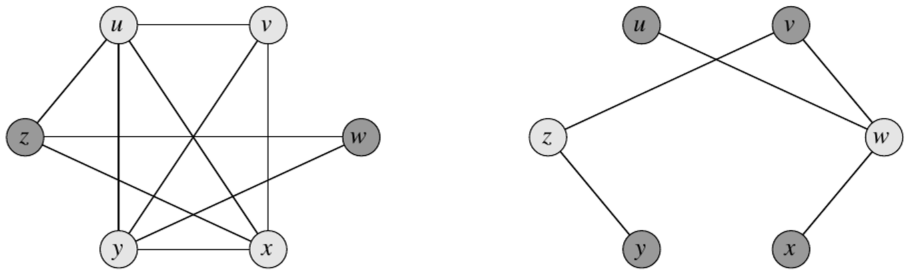


FIGURE 3 – Soit le graphe de gauche dont on souhaite connaître s'il contient une clique de taille $k = 4$. On effectue la couverture de sommets du graphe dual. Si cette couverture contient $|S| - k] = 2$ sommets alors le graphe initial contient bien une clique de taille $k = 4$.

Vérifiez ce qu'il se passe si ce même graphe de gauche n'a pas d'arrête reliant u et v.

2 Le problème $2\|C_{\max}$ est-il NPC ?

On considère le problème $2\|C_{\max}$ qui consiste à répartir *au mieux* N tâches sur deux machines. Chaque tâche i a une durée opératoire p_i. On note

$$P = \sum_{i=1}^{N} p_i$$

De plus, on connaît une PD de complexité en temps en $O(NP)$

- Montrez que ce problème est polynomial en temps si le codage est unaire,
- Montrez que ce problème est NPC si le codage est binaire
  - Avant tout, il faut transformer le problème d'*optimisation* $2\|C_{\max}$ en un problème de *décision* $\mathbf{DECISION}\left(\mathbf{2}\|\mathbf{C}_{\max}\right)$,
  - Ensuite utiliser le fait que le problème **SUBSETSUM** est connu NPC pour établir que $\mathbf{DECISION}\left(\mathbf{2}\|\mathbf{C}_{\max}\right)$ l'est également.

Une instance est codée sur $(N + P)$ bits en unaire. Ce qui implique que la complexité $O(NP)$ est polynomiale pour ce codage.

La version **décision** revient à fixer*a priori* un coût C et à vérifier que l'instance donnée $I = \{p_1, \ldots, p_N\}$ peut être ordonnancée avec un coût $C_{\max} \leq C$.

Définition SUBSETSUM. *Soit un ensemble fini $S \subset \mathbb{N}$ et un objectif $t \in \mathbb{N}$. On veut savoir s'il existe un sous-ensemble $S' \subseteq S$ dont la somme des éléments est t.*

Ensuite on observe que le problème de somme de sous-ensembles SUMSUBSET se réduit au problème de décision $\mathbf{DECISION}\left(\mathbf{2}\|\mathbf{C}_{\max}\right)$.

Puisque **SUBSETSUM** est NPC, $\mathbf{DECISION}\left(\mathbf{2}\|\mathbf{C}_{\max}\right)$ l'est également.

Remarque et remerciement. Je n'oublie pas de remercier Cormen pour les figures prises de son livre (3ed) Je vous invite évidemment à lire son chapitre dédié à la complexité qui a le très grand mérite (car pas si courant que ça) d'être clair et pédagogique.